STEM Fusion

Introduction to Computational Thinking and Python Programming

PYTHON DATA TYPES IN DEPTH

Integers & Floats · Strings · Booleans · Lists · Tuples · Dictionaries

Instructor: Sarom Leang, PhD
Specialist: Daniela Garcia Galindo
Innovation Hall

Pronouns

Instructor

Sarom Leang, Ph.D.

Pronouns: Teacher / Instructor

Specialist

Daniela Garcia Galindo

Pronouns: Daniela

Schedule

10:00 AM - 10:15 AM	Homeroom
10:15 AM - 11:35 AM	G1
11:40 AM - 12:35 PM	Lunch
12:40 PM - 02:00 PM	G2

Student Expectations

NO FOOD

NO DRINKS (on the table)

Be respectful to individuals and property

Be open to learning

Be open to not understanding

Be patient with yourself

Ask questions

Explore

Embrace failure

Course Overview

This course introduces the fundamental building blocks of computational thinking and computer programming using the Python language.

Upon successful completion of this course, students will be able to:

- Improve their problem-solving skills
- Write, read, and execute Python code using basic data types and operators

Course Overview (cont.)

Exploratory Topics

- ~~Session #1 – November 1, 2025~~
  - ~~Entrance Survey~~
  - ~~How Computers Work~~
- ~~Session #2 – December 6, 2025~~
  - ~~Parallel Computing~~
- ~~Session #3 – January 31, 2025~~
  - ~~Generative Artificial Intelligence~~
- Session #4 – March 7, 2026
  - Generative Artificial Intelligence
- Session #5 – April 11, 2026
  - Future of Computing (Quantum)
  - Exit survey

Topics

- Algorithms and Problem Solving
- Debugging and Troubleshooting
- Loops
- Algorithms and Syntax
- Variables and Conditionals
- Variable Arithmetic
- Conditionals (If/Else)
- Compound Conditionals

What Are We Doing Today?

1	2	3
		
Integers & Floats	Strings In Depth	Booleans
4	**5**	**6**
		
Lists In Depth	Tuples	Dictionaries

Integers & Floats

The two number types — and when each one matters

int vs float — Two Kinds of Number

*Python has two numeric types — **int** for whole numbers, **float** for decimals.*
They behave differently in math and memory.

int — whole numbers

```
jersey = 23          # int
points = 842         # int
games = 82           # int

print(type(jersey))
# <class 'int'>

print(jersey + points)
# 865  (exact, no decimal)

# int division:
print(10 // 3)   # 3
print(10 % 3)    # 1  (remainder)
```

float — decimal numbers

```
ppg = 28.7           # float
body_temp = 98.6     # float
pi = 3.14159         # float

print(type(ppg))
# <class 'float'>

# Regular / always gives float:
print(10 / 3)
# 3.3333333333333335

print(10 / 2)
# 5.0  (float even when exact!)
```

Converting Between int and float

Use int() and float() to convert values between types — critical when handling user input or mixing data sources.

int() and float() conversions

```
# float → int (TRUNCATES, not rounds)
print(int(9.9))    # 9
print(int(3.1))    # 3
print(int(-2.8))   # -2

# int → float
print(float(5))    # 5.0
print(float(100))  # 100.0

# string → number (from input)
age_str = '17'
age = int(age_str)
print(age + 1)     # 18
```

Real world: calculating a grade

```
# input() always returns str!
total_str = input('Total points: ')
max_str = input('Max possible: ')

# Convert to int before math
total = int(total_str)
max_pts = int(max_str)

# Use float division for %
percent = (total / max_pts) * 100
print('Score:', round(percent, 1), '%')
# Score: 87.5 %
```

⚠️ Floating Point Surprises

Computers store decimals in binary — and some numbers can't be stored exactly.
Here's what that looks like in practice.

😱 The surprise

```
print(0.1 + 0.2)
# 0.30000000000000004
# NOT 0.3 !

print(0.1 + 0.2 == 0.3)
# False
```

✅ The fix: round()

```
result = 0.1 + 0.2
print(round(result, 2))
# 0.3  ✓

# round(number, decimal_places)
print(round(3.14159, 2)) # 3.14
print(round(9.876, 1))   # 9.9
```

When does this matter?

💰 Money calculations · 📐 Geometry checks · 📊 Data comparisons

Fix: always use round() before displaying or comparing float results.

⚡ OPTIONAL CHALLENGE — Integers & Floats

Pen & Paper

A streaming service charges $14.99/month. A user has been subscribed for 14 months.

On paper, answer:
1. What is the total amount paid? (float × int)
2. What Python type does 14.99 * 14 return?
3. If you want to display it as '$209.86', what function do you use?
4. What does int(14.99) give? Why?

Bonus: the service rounds all bills to 2 decimal places. Write the Python expression that computes and rounds the total.

Try it in IDLE

```
# Try it in IDLE!
price = 14.99
months = 14

total = price * months
print(total)
# What do you get?

print(type(total))
# int or float?

print(round(total, 2))
# How does this look?

print(int(price))
# What does this give?

# Bonus:
print('Total: $' + str(round(price * months, 2)))
```

Work individually • There's no wrong answer — just think it through!

✅ ANSWERS — Integers & Floats

🖊️ Pen & Paper

A streaming service charges $14.99/month. A user has been subscribed for 14 months.

On paper, answer:
1. What is the total amount paid? (float × int)
2. What Python type does 14.99 * 14 return?
3. If you want to display it as '$209.86', what function do you use?
4. What does int(14.99) give? Why?

Bonus: the service rounds all bills to 2 decimal places. Write the Python expression that computes and rounds the total.

💻 IDLE Output

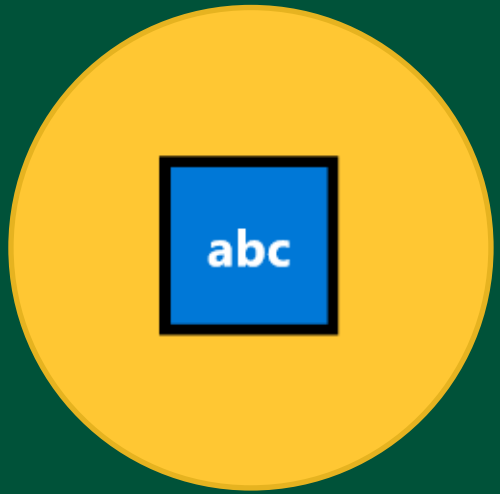
```
>>> price = 14.99
>>> months = 14
>>> total = price * months
>>> print(total)
209.86

>>> print(type(total))
<class 'float'>

>>> print(round(total, 2))
209.86

>>> print(int(price))
14
# Truncates — NOT 15!

# Bonus:
>>> print('Total: $' + str(round(price * months, 2)))
Total: $209.86
```

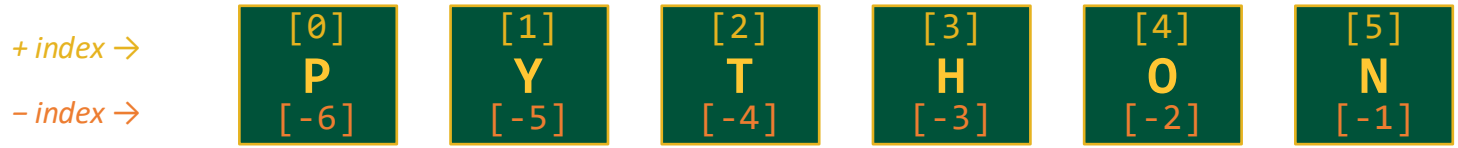


Strings In Depth

Slice them, search them, format them — text is everywhere

Indexing & Slicing Strings ✂️

```
text = 'PYTHON'
```



Indexing — single characters

```
text = 'PYTHON'
print(text[0])   # P
print(text[3])   # H
print(text[-1])  # N  (last)
print(text[-2])  # O  (second to last)
```

Slicing — ranges of characters

```
text = 'PYTHON'
print(text[0:3])  # PYT
print(text[2:])   # THON
print(text[:4])   # PYTH
print(text[::2])  # PTO  (every 2nd)
print(text[::-1]) # NOHTYP (reversed!)
```

Essential String Methods 🛠️

.upper() / .lower()

```
name = 'Taylor Swift'
print(name.upper())
# TAYLOR SWIFT
print(name.lower())
# taylor swift
```

.strip() / .lstrip() / .rstrip()

```
raw = '  hello world  '
print(raw.strip())
# 'hello world'
# Removes whitespace from edges
```

.replace(old, new)

```
title = 'Not Like Us'
clean = title.replace(' ', '_')
print(clean)
# Not_Like_Us
```

.find(substring)

```
bio = 'I love Python!'
print(bio.find('Python'))
# 7  (index where found)
print(bio.find('Java'))
# -1  (not found)
```

.count(substring)

```
lyric = 'na na na batman'
print(lyric.count('na'))
# 3
```

.startswith() / .endswith()

```
url = 'https://spotify.com'
print(url.startswith('https'))
# True
print(url.endswith('.com'))
# True
```

split(), join(), and f-strings ✨

.split()

Splits a string into a list of parts

```
sentence = 'Anti-Hero by
Taylor Swift'
words = sentence.split()
print(words)
# ['Anti-Hero', 'by',
#  'Taylor', 'Swift']

# Split on a delimiter:
csv = '85,90,72,96'
scores = csv.split(',')
print(scores)
# ['85','90','72','96']
```

.join()

Joins a list of strings into one

```
words = ['Not', 'Like', 'Us']
result = ' '.join(words)
print(result)
# Not Like Us

# Join with any separator:
tags = ['python','code','fun']
print(', '.join(tags))
# python, code, fun

print('-'.join(tags))
# python-code-fun
```

f-strings ★

The modern way to build strings

```
name = 'Jordan'
score = 95

# Old way (messy):
print('Hi ' + name + '!')

# f-string (clean!):
print(f'Hi {name}!')
# Hi Jordan!

print(f'{name} scored
{score}%')
# Jordan scored 95%

print(f'Score: {score *
1.1:.1f}')
# Score: 104.5
```

⚡ OPTIONAL CHALLENGE — Strings In Depth

Pen & Paper

Given this string:
handle = '@kendrick_lamar'

On paper, write the Python expressions to:
1. Print the handle in ALL CAPS
2. Print just the username without the @ symbol (use slicing)
3. Check if the handle starts with '@' (True/False)
4. Replace underscores with spaces and print the result
5. Count how many times the letter 'a' appears

Bonus: use an f-string to print:
'Artist: kendrick lamar (handle: @kendrick_lamar)'

Try it in IDLE

```
handle = '@kendrick_lamar'

# 1. All caps:
print(handle.upper())

# 2. Without @:
print(handle[1:])

# 3. Starts with @?
print(handle.startswith('@'))

# 4. Replace _ with space:
print(handle.replace('_', ' '))

# 5. Count 'a':
print(handle.count('a'))

# Bonus:
name = handle[1:].replace('_', ' ')
print(f'Artist: {name} (handle: {handle})')
```

Work individually • There's no wrong answer — just think it through!

ANSWERS — Strings In Depth

Pen & Paper

Given this string:
handle = '@kendrick_lamar'

On paper, write the Python expressions to:
1. Print the handle in ALL CAPS
2. Print just the username without the @ symbol (use slicing)
3. Check if the handle starts with '@' (True/False)
4. Replace underscores with spaces and print the result
5. Count how many times the letter 'a' appears

Bonus: use an f-string to print:
'Artist: kendrick lamar (handle: @kendrick_lamar)'

IDLE Output

```
>>> handle = '@kendrick_lamar'
>>> print(handle.upper())
@KENDRICK_LAMAR
>>> print(handle[1:])
kendrick_lamar
>>> print(handle.startswith('@'))
True
>>> print(handle.replace('_', ' '))
@kendrick lamar
>>> print(handle.count('a'))
3

# Bonus:
>>> name = handle[1:].replace('_', ' ')
>>> print(f'Artist: {name} (handle: {handle})')
Artist: kendrick lamar (handle: @kendrick_lamar)
```

Booleans

True, False — and everything in between

The bool Type — and bool() Conversion

In Python, True equals 1 and False equals 0.
Any value can be converted to bool — and Python does this automatically in if statements.

bool is a subtype of int

```
print(True == 1)    # True
print(False == 0)   # True
print(True + True)  # 2
print(True * 5)     # 5

# Type is still bool:
print(type(True))   # <class 'bool'>

# Counting trick — sum of bools:
results = [True, False, True, True]
print(sum(results)) # 3  (three Trues)
```

bool() conversion

```
# Any value → True or False
print(bool(1))     # True
print(bool(0))     # False
print(bool('hi'))  # True
print(bool(''))    # False
print(bool([1,2])) # True
print(bool([]))    # False
print(bool(None))  # False

# Python uses this automatically:
name = input('Your name: ')
if name:  # same as: if bool(name):
    print(f'Hello, {name}!')
```

Truthy and Falsy Values

Python treats some values as True and others as False when used in if statements — even without comparison operators.

❌ Falsy — behave like False

`False`	the boolean False
`0 / 0.0`	zero integers and floats
`''`	empty string
`[]`	empty list
`()`	empty tuple
`{}`	empty dict
`None`	the absence of a value

✅ Truthy — behave like True

```
# Practical examples:

playlist = ['Song A', 'Song B']
if playlist:            # truthy
    print('Playlist ready!')

username = ''
if not username:        # falsy
    print('Please enter a name')

result = None
if result is None:      # explicit
    print('No result yet')
```

⚡ OPTIONAL CHALLENGE — Booleans

Pen & Paper

Without running the code, predict True or False for each:

1. bool(0)
2. bool('False')
3. bool([])
4. bool([0])
5. bool('')
6. bool(' ')
7. True + True + False
8. not bool(None)

Bonus: a shopping cart function:

```
def has_items(cart):
    return bool(cart)
```

What does has_items([]) return?
What does has_items(['milk']) return?

Try it in IDLE

```
# Verify in IDLE!
print(bool(0))
print(bool('False'))
print(bool([]))
print(bool([0]))
print(bool(''))
print(bool(' '))
print(True + True + False)
print(not bool(None))

# Bonus:
def has_items(cart):
    return bool(cart)

print(has_items([]))
print(has_items(['milk']))
```

Work individually • There's no wrong answer — just think it through!

ANSWERS — Booleans

Pen & Paper

Without running the code, predict True or False for each:

1. bool(0)
2. bool('False')
3. bool([])
4. bool([0])
5. bool('')
6. bool(' ')
7. True + True + False
8. not bool(None)

Bonus: a shopping cart function:

```
def has_items(cart):
    return bool(cart)
```

What does has_items([]) return?
What does has_items(['milk']) return?

IDLE Output

```
>>> print(bool(0))
False
>>> print(bool('False'))
True
>>> print(bool([]))
False
>>> print(bool([0]))
True
>>> print(bool(''))
False
>>> print(bool(' '))
True
>>> print(True + True + False)
2
>>> print(not bool(None))
True

# Bonus:
>>> print(has_items([]))
False
>>> print(has_items(['milk']))
True
```

Lists In Depth

Mutable, ordered, powerful — the workhorse of Python data

Lists: Indexing, Slicing & Mutability

Lists use the same [index] and [start:stop] syntax as strings — but unlike strings, lists are mutable: you can change them in place.

Indexing and slicing

```
queue = ['Song A', 'Song B',
         'Song C', 'Song D']

print(queue[0])    # Song A
print(queue[-1])   # Song D
print(queue[1:3])  # ['Song B', 'Song C']
print(queue[:2])   # ['Song A', 'Song B']
print(len(queue))  # 4
```

Mutability — lists can change

```
queue = ['Song A', 'Song B', 'Song C']

# Change an item in place:
queue[1] = 'Song X'
print(queue)
# ['Song A', 'Song X', 'Song C']

# Strings CANNOT do this:
name = 'Python'
name[0] = 'J'  # TypeError!

# Lists CAN. Strings CANNOT.
# This is the key difference.
```

Essential List Methods 

.append(item)

```
songs = ['Anti-Hero']
songs.append('Flowers')
print(songs)
# ['Anti-Hero', 'Flowers']
```

.insert(index, item)

```
songs.insert(0, 'Kill Bill')
print(songs)
# ['Kill Bill', 'Anti-Hero',
#  'Flowers']
```

.remove(item)

```
songs.remove('Flowers')
print(songs)
# ['Kill Bill', 'Anti-Hero']
# Removes FIRST match by value
```

.pop(index)

```
removed = songs.pop(0)
print(removed)  # Kill Bill
print(songs)    # ['Anti-Hero']
# Returns removed item
```

.sort() / .reverse()

```
nums = [3, 1, 4, 1, 5]
nums.sort()
print(nums)  # [1, 1, 3, 4, 5]
nums.reverse()
print(nums)  # [5, 4, 3, 1, 1]
```

in / .index(item)

```
songs = ['Anti-Hero', 'Flowers']
print('Flowers' in songs)     # True
print('Levitating' in songs)  # False
print(songs.index('Flowers')) # 1
```

⚠ List Copying — The = Trap

= does NOT copy a list — it creates a second name pointing to the same list. Changes to one affect both!

❌ The trap — = shares the list

```
original = [1, 2, 3]
copy = original    # NOT a copy!

copy.append(4)

print(original)
# [1, 2, 3, 4]  😱
# original changed too!

# Both names point to
# THE SAME list in memory.
```

✅ Three ways to copy properly

```
original = [1, 2, 3]

# Method 1: .copy()
copy1 = original.copy()

# Method 2: slice
copy2 = original[:]

# Method 3: list()
copy3 = list(original)

copy1.append(99)
print(original) # [1, 2, 3]  ✓
print(copy1)    # [1, 2, 3, 99]
```

⚡ OPTIONAL CHALLENGE — Lists In Depth

Pen & Paper

Start with this list:
watchlist = ['Inception', 'Dune', 'The Matrix',
'Interstellar', 'Oppenheimer']

On paper, write the Python lines to:
1. Add 'Arrival' to the end
2. Insert 'Tenet' at position 2
3. Remove 'The Matrix' by value
4. Print the last 3 titles using slicing
5. Sort the list alphabetically and print it

Bonus: make a TRUE copy of the sorted list, then reverse ONLY the copy. Print both to prove they're independent.

Try it in IDLE

```
watchlist = ['Inception', 'Dune', 'The Matrix',
             'Interstellar', 'Oppenheimer']

# 1. Add to end:
watchlist.append('Arrival')
# 2. Insert at position 2:
watchlist.insert(2, 'Tenet')
# 3. Remove by value:
watchlist.remove('The Matrix')
# 4. Last 3:
print(watchlist[-3:])
# 5. Sort:
watchlist.sort()
print(watchlist)
# Bonus: true copy then reverse:
reversed_list = watchlist.copy()
reversed_list.reverse()
print(watchlist)
print(reversed_list)
```

Work individually • There's no wrong answer — just think it through!

ANSWERS — Lists In Depth

Pen & Paper

Start with this list:
watchlist = ['Inception', 'Dune', 'The Matrix',
'Interstellar', 'Oppenheimer']

On paper, write the Python lines to:
1. Add 'Arrival' to the end
2. Insert 'Tenet' at position 2
3. Remove 'The Matrix' by value
4. Print the last 3 titles using slicing
5. Sort the list alphabetically and print it

Bonus: make a TRUE copy of the sorted list, then reverse ONLY the copy. Print both to prove they're independent.

IDLE Output

```
>>> watchlist.append('Arrival')
>>> watchlist.insert(2, 'Tenet')
>>> watchlist.remove('The Matrix')
>>> print(watchlist[-3:])
['Interstellar', 'Oppenheimer', 'Arrival']
>>> watchlist.sort()
>>> print(watchlist)
['Arrival', 'Dune', 'Inception',
'Interstellar', 'Oppenheimer', 'Tenet']

# Bonus:
>>> rev = watchlist.copy()
>>> rev.reverse()
>>> print(watchlist)
['Arrival', 'Dune', ...]
>>> print(rev)
['Tenet', 'Oppenheimer', ...]
```

Tuples

Immutable sequences — when the data should never change

Tuples — Immutable and Ordered 📌

Tuples are like lists, but immutable — once created, they cannot be changed. Use () instead of [].

Creating and accessing tuples

```
# Create with ()
location = (40.7128, -74.0060)  # NYC
rgb_red  = (255, 0, 0)
days     = ('Mon','Tue','Wed')

# Access same as list:
print(location[0])  # 40.7128
print(rgb_red[-1])  # 0
print(days[1:])     # ('Tue', 'Wed')

# Trying to change → TypeError:
location[0] = 99    # TypeError!
```

When to use tuple vs list

Use tuple when:	Use list when:
Data is fixed	Data may change
GPS coordinates	Shopping cart
RGB color values	Playlist songs
Days of the week	Search results
Function return values	Collected user inputs

Tuple Packing & Unpacking

Unpacking assigns each item in a tuple to its own variable in one clean line.

Packing and unpacking

```
# Packing — values → tuple
point = (10, 25)
color = (255, 128, 0)

# Unpacking — tuple → variables
x, y = point
print(x)  # 10
print(y)  # 25

r, g, b = color
print(f'R:{r} G:{g} B:{b}')
# R:255 G:128 B:0

# Classic swap trick:
a, b = 5, 10
a, b = b, a
print(a, b)  # 10  5
```

Functions returning multiple values

```
# Python returns a tuple!
def min_max(numbers):
    return min(numbers), max(numbers)

scores = [85, 42, 91, 67, 73]

# Unpack the return value:
lowest, highest = min_max(scores)
print('Low:', lowest)   # 42
print('High:', highest) # 91

# Or keep as tuple:
result = min_max(scores)
print(type(result))  # <class 'tuple'>
print(result)        # (42, 91)
```

⚡ OPTIONAL CHALLENGE — Tuples

Pen & Paper

A pixel in an image is stored as an RGB tuple: (red, green, blue) where each value is 0–255.

Given:
pixel = (180, 50, 220)

On paper:
1. Unpack pixel into r, g, b
2. What is the type of pixel?
3. Try to change pixel[0] to 100. What happens?
4. Write a function called brightness(pixel) that returns the average of r, g, b as a float

Bonus: write a function called is_dark(pixel) that returns True if the brightness is below 128.

Try it in IDLE

```
pixel = (180, 50, 220)

# 1. Unpack:
r, g, b = pixel
print(r, g, b)
# 2. Type:
print(type(pixel))
# 3. Try to change:
pixel[0] = 100  # What happens?
# 4. Brightness function:
def brightness(pixel):
    r, g, b = pixel
    return (r + g + b) / 3
print(brightness(pixel))
# Bonus:
def is_dark(pixel):
    return brightness(pixel) < 128
print(is_dark(pixel))
print(is_dark((10, 10, 10)))
```

Work individually • There's no wrong answer — just think it through!

ANSWERS — Tuples

Pen & Paper

Start with this list:
watchlist = ['Inception', 'Dune', 'The Matrix',
'Interstellar', 'Oppenheimer']

On paper, write the Python lines to:
1. Add 'Arrival' to the end
2. Insert 'Tenet' at position 2
3. Remove 'The Matrix' by value
4. Print the last 3 titles using slicing
5. Sort the list alphabetically and print it

Bonus: make a TRUE copy of the sorted list, then reverse ONLY the copy. Print both to prove they're independent.

IDLE Output

```
>>> r, g, b = pixel
>>> print(r, g, b)
180 50 220
>>> print(type(pixel))
<class 'tuple'>
>>> pixel[0] = 100
TypeError: 'tuple' object does
not support item assignment
>>> def brightness(pixel):
...     r, g, b = pixel
...     return (r + g + b) / 3
>>> print(brightness(pixel))
150.0
>>> def is_dark(pixel):
...     return brightness(pixel) < 128
>>> print(is_dark(pixel))
False
>>> print(is_dark((10, 10, 10)))
True
```

Dictionaries

Key-value pairs — look anything up by name, instantly

Creating and Accessing Dictionaries 📖

A dictionary stores key-value pairs. Look up any value instantly using its key — like a real-world dictionary.

Creating and reading

```
# Create with {key: value}
pokemon = { 'name':   'Pikachu',
            'type':   'Electric',
            'hp':     35,
            'speed':  90
}

# Access by key:
print(pokemon['name'])   # Pikachu
print(pokemon['hp'])     # 35

# Safe access with .get():
print(pokemon.get('attack'))  # None
# (no crash if key missing)
```

Adding, updating, deleting

```
pokemon = {'name': 'Pikachu', 'hp': 35}

# Add a new key:
pokemon['attack'] = 55
# Update existing key:
pokemon['hp'] = 50
# Delete a key:
del pokemon['attack']
# Check if key exists:
print('hp' in pokemon)    # True
print('speed' in pokemon) # False

print(pokemon)
# {'name': 'Pikachu', 'hp': 50}
```

Dictionary Methods & Looping

Three views into a dictionary — and the key loop pattern that ties them all together.

.keys()

```
pokemon = {'name':'Pikachu',
           'type':'Electric',
           'hp': 35}

for key in pokemon.keys():
    print(key)
# name
# type
# hp
```

.values()

```
for val in pokemon.values():
    print(val)
# Pikachu
# Electric
# 35
```

.items()

```
for key, val in
pokemon.items():
    print(key, ':', val)
# name : Pikachu
# type : Electric
# hp   : 35
```

Real-world: printing a character stat card:

```
stats = {'HP': 35, 'ATK': 55, 'SPD': 90}
for stat, value in stats.items():
    print(f'{stat:>4}: {value}')
```

⚡ OPTIONAL CHALLENGE — Dictionaries

Pen & Paper

Build a character profile dictionary for your favourite game, movie, or show character.

Your dict must have at least 5 keys:
name, type/class, level, power, is_hero

Then on paper:
1. Access and print the character's name
2. Increase the level by 1
3. Add a new key: 'catchphrase'
4. Use a for loop with .items() to print all stats in the format 'key: value'

Bonus: write a function called stat_summary(character) that prints all keys and values using .items()

Try it in IDLE

```
character = { 'name':     'Spider-Man',
              'type':     'Hero',
              'level':    10,
              'power':    9.5,
              'is_hero': True }
# 1. Access name:
print(character['name'])
# 2. Level up:
character['level'] += 1
# 3. Add catchphrase:
character['catchphrase'] = 'With great power...'
# 4. Print all stats:
for key, val in character.items():
    print(f'{key}: {val}')

# Bonus:
def stat_summary(c):
    for k, v in c.items():
        print(f'{k}: {v}')
```

Work individually • There's no wrong answer — just think it through!

ANSWERS — Dictionaries

Pen & Paper

Build a character profile dictionary for your favourite game, movie, or show character.

Your dict must have at least 5 keys:
name, type/class, level, power, is_hero

Then on paper:
1. Access and print the character's name
2. Increase the level by 1
3. Add a new key: 'catchphrase'
4. Use a for loop with .items() to print all stats in the format 'key: value'

Bonus: write a function called stat_summary(character) that prints all keys and values using .items()

IDLE Output

```
>>> print(character['name'])
Spider-Man
>>> character['level'] += 1
>>> print(character['level'])
11
>>> character['catchphrase'] = 'With great power...'
>>> for key, val in character.items():
...     print(f'{key}: {val}')
name: Spider-Man
type: Hero
level: 11
power: 9.5
is_hero: True
catchphrase: With great power...

# Bonus:
>>> stat_summary(character)
# same output as above
```

Python Data Types — Unlocked!

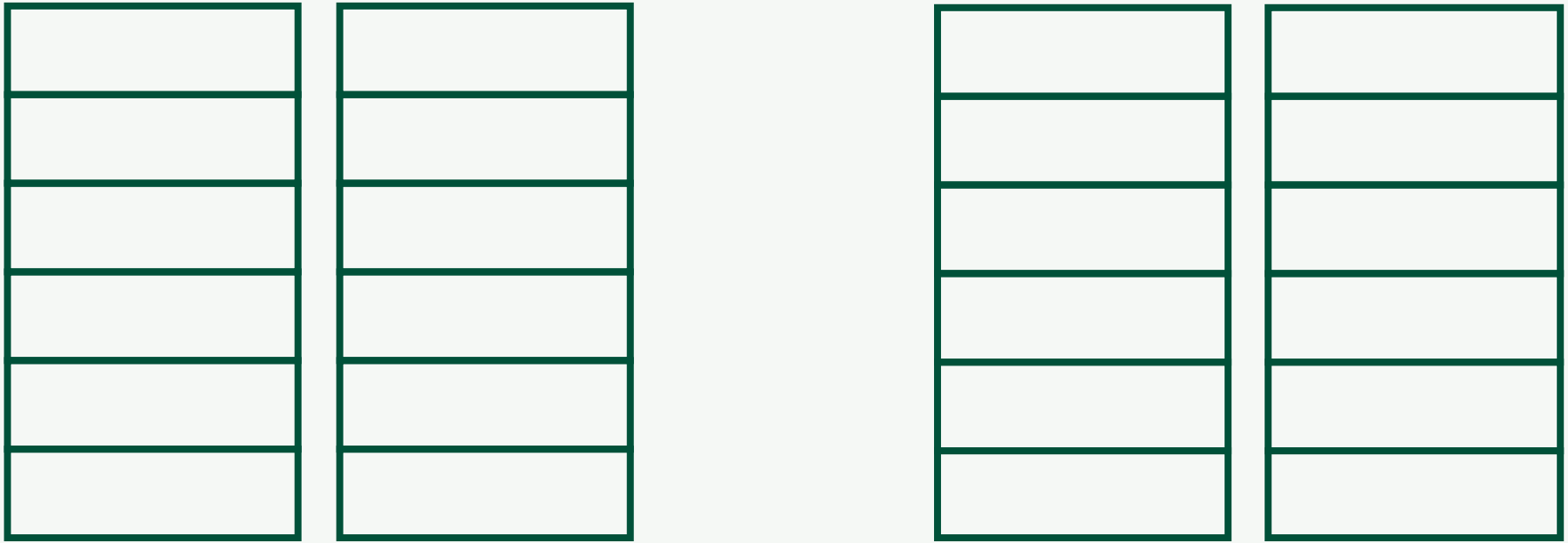
You now know Python's core data types — deeply:

- 🔢 int & float — and when to use each
- 🔤 Strings — slice, search, format
- ✅ Booleans — truthy, falsy, bool()
- 📋 Lists — mutable, methods, copies
- 📌 Tuples — immutable, unpack, return
- 📖 Dictionaries — key-value, .items()

What's next: Classes & Objects · File I/O · Error Handling · APIs 🚀

Each data type is a tool. Great programmers know which tool to reach for.

Innovation Computer Issues



Front Screen